

PiXiEEDraw

システム詳細資料

GPT 記事作成用の基礎資料 / 2026 年 6 月版

実装確認済み・開発経緯・将来構想を分けて整理

区分	意味
実装確認済み	公開画面、保存済みコード、既存機能から確認できる内容
開発経緯上で確認	同期ログ、修正方針、既知不具合などから確認できる内容
構想・推奨設計	将来案または説明用モデル。現在の機能として断定しない内容

文書の目的

本資料は、PiXiEEDraw について GPT などの文章生成 AI に記事を書かせる際の、基礎情報・技術説明・表現上の注意点をまとめたものです。

単なる宣伝文ではなく、PiXiEEDraw がどのような構成で動き、どのように描画・保存・共有・復帰・拡張を行うのかを、公開可能な範囲で整理しています。

本資料では、情報を次の 3 区分で扱います。

- 実装確認済み：画面、公開 HTML、保存済みコード、既存機能から確認できる内容
- 会話・開発経緯上で確認：開発時のログ、修正方針、既知不具合などから確認できる内容
- 構想・推奨設計：将来案、未確定仕様、記事内で断定してはいけない内容

記事化する際は、実装確認済みの内容を中心にし、構想は「今後の展望」「検討中」として扱ってください。

1. PiXiEEDraw の概要

PiXiEEDraw は、インストールせずブラウザ上で利用できるドット絵制作ツールです。PC、タブレット、スマートフォンに対応し、通常のピクセル描画だけでなく、レイヤー、フレームアニメーション、オニオンスキン、複数形式での書き出し、リアルタイム共同編集を備えています。

PiXiEEDraw の特徴は、単なる「ブラウザで描けるドット絵エディター」にとどまらない点です。端末内への自動保存、PWA としての利用、共有プロジェクト、操作履歴を使った同期、ローカル拡張による補助機能の追加などを組み合わせ、制作環境そのものを Web 上に構築しています。

中核となる価値は次の 4 点です。

1. ブラウザですぐに使える
2. PC とスマートフォンの両方で制作できる
3. フレームアニメーションと複数レイヤーを扱える
4. 複数人で同じ作品をリアルタイム編集できる

長期的には、Aseprite のような本格的な制作機能を持ちながら、ブラウザ、スマートフォン、共同編集、外部拡張という Web ならではの強みを持つエディターを目指しています。

2. システム全体構成

PiXiEEDraw は、概念的には次の層で構成されています。

- 表示・操作 UI 層
- プロジェクト状態管理層
- 描画・レンダリング層
- ツール処理層
- レイヤー・フレーム管理層
- パレット・色管理層
- ファイル入出力層
- 端末内保存・復元層
- 共有プロジェクト同期層
- PWA・モバイル復帰層
- ローカル拡張実行層
- 認証・分析・広告などの周辺機能

ユーザーがキャンバスへ線を描くと、ポインター入力ツール処理へ渡り、現在選択中のフレームとレイヤーのピクセルデータが更新されます。更新後はキャンバス表示、プレビュー、履歴、端末内保存が更新されます。共有モード中であれば、ローカル変更を共有操作として送信し、他の参加者側でも同じ操作を適用します。

重要なのは、画面に表示されている Canvas だけを保存対象にするのではなく、プロジェクト、フレーム、レイヤー、パレット、設定などを一つの編集状態として扱う点です。

3. フロントエンドと表示構造

PiXiEEDraw は HTML、CSS、JavaScript を中心とした Web アプリケーションです。描画部分には Canvas を使用し、マウス、ペン、タッチ入力を統一的に扱うために Pointer Events 系の入力処理を利用する構成です。

画面は大きく次の領域に分かれます。

- 起動画面
- メインキャンバス
- ツール選択領域
- カラー・パレット領域
- レイヤー・フレーム領域
- 設定・書き出し領域
- プレビュー領域
- 共有モード関連 UI
- ローカル拡張パネル

スマートフォンでは表示幅が限られるため、すべてのパネルを同時表示するのではなく、ツール、カラー、フレームとレイヤー、設定、ヘルプなどをタブやパネル切り替えで表示します。

キャンバスは CSS 上の表示サイズと内部ピクセルサイズを分けて扱います。見た目は大きく拡大しても、内部では 1 ピクセル単位のデータを維持し、`image-rendering: pixelated` に相当する表示によって輪郭をぼかさず確認できる設計です。

4. 起動・プロジェクト復元システム

起動時には、主に次の選択肢が表示されます。

- 前回の続きから再開
- 新規作成
- ファイルを開く

端末内に自動保存されたプロジェクトがある場合、最近のプロジェクトを一覧で確認し、再開できます。新規作成時にはプロジェクト名、キャンバス幅、キャンバス高さなどを設定します。確認済みの仕様ではキャンバスサイズは 1~512 ピクセル、初期値は 32×32 です。

起動処理では、単に最後の画面を表示するだけでなく、次の確認が必要になります。

5. 端末内保存データの有無を確認
6. 保存データの形式とバージョンを確認
7. プロジェクト状態を復元
8. レイヤー、フレーム、パレット、選択状態などを再構築
9. 共有プロジェクトであればサーバー側状態と再同期

10. 描画 UI を有効化

共有プロジェクトの復元は通常のローカル復元より複雑です。ローカルに残っている状態が最新とは限らないため、共有プロジェクト ID と Revision を確認し、未取得操作を適用してから編集可能状態へ戻す必要があります。

5. プロジェクトの概念データ構造

内部の厳密な変数名や保存形式は記事で断定せず、概念として次の構造を説明するのが安全です。

Project

- プロジェクト ID
- プロジェクト名
- キャンバス幅・高さ
- カラーモード
- パレット
- フレーム一覧
- 現在選択中のフレーム
- 現在選択中のレイヤー
- 描画・表示設定
- アニメーション設定
- 保存日時・更新日時
- 共有プロジェクト情報

Frame

- フレーム ID
- 表示順
- フレーム内レイヤー一覧
- 表示時間または FPS に関する情報

Layer

- レイヤー ID
- レイヤー名
- 表示順
- 表示・非表示
- 不透明度
- 合成モード
- ピクセルデータ

Palette

- 色一覧
- 現在色
- プリセットまたはカスタムの区分
- インデックスカラー時の色番号

SharedProject

- 共有プロジェクト ID
- 招待情報
- マスター・参加者の状態
- 現在 Revision
- 接続状態
- 未送信・未適用操作

Operation

- 操作 ID
- 対象プロジェクト
- Revision
- 操作者
- 操作種別
- 対象フレーム・レイヤー
- 変更データ
- 作成時刻

これは説明用の概念モデルです。記事内で「内部データベースに必ずこの項目名で保存している」とは書かないでください。

6. 描入力とツル理

描画操作は、マウス、スタイラス、タッチを Pointer Events で受け取り、キャンバス上の座標をプロジェクト内のピクセル座標へ変換して処理します。

基本的な処理の流れは次のとおりです。

11. pointerdown で描画開始位置を取得
12. 表示倍率とキャンバス位置からピクセル座標へ変換
13. 現在のツール、色、ブラシサイズを確認
14. 対象レイヤーのピクセルデータを変更
15. pointermove 中に連続座標を処理
16. キャンバスとプレビューを再描画
17. Undo 用履歴を確定
18. 自動保存を予約

19. 共有モードでは変更操作を送信

20. pointerup または pointercancel で操作を終了

ブラウザの表示倍率や CSS サイズが変化しても正しいセルへ描画するため、getBoundingClientRect で得た表示領域と Canvas 内部サイズの比率を使って座標を補正する必要があります。

主な描画・編集機能として、ペン、消しゴム、塗りつぶし、スポイト、選択、選択範囲の移動、コピー、貼り付け、拡大縮小、キャンバス移動、Undo、Redo、ブラシサイズ変更などがあります。

ミラー描画では、基準軸と対象方向を設定し、一度の入力を左右または上下の対応座標へ複製します。対称キャラクター、正面顔、アイコン、模様などの制作効率を上げる機能です。

7. 描画データとレンダリング

PiXiEEDraw では、編集用のピクセルデータと画面表示を分けて考える必要があります。画面上の Canvas は編集状態を表示する出力先であり、プロジェクトの本体はフレーム・レイヤーごとのピクセル情報です。

再描画時には、概念的に次の処理を行います。

21. 現在フレームのレイヤーを下から順に取得
22. 非表示レイヤーを除外
23. レイヤーごとの不透明度と合成モードを適用
24. ピクセルデータを合成
25. オニオンスキンが有効なら前後フレームを補助表示
26. 選択範囲やミラー軸など編集用オーバーレイを表示
27. メインキャンバスと等倍プレビューを更新

小窓プレビューは、拡大表示した作業キャンバスだけでは判断しづらい等倍時の見え方を確認するためのものです。ドット絵では 1 ピクセルの差が全体の印象を変えるため、編集用拡大表示と等倍プレビューを同時に持つ意味があります。

8. Undo・Redo と操作確定

Undo・Redo は、ピクセル単位の変更を無制限にそのまま記録するのではなく、一回のストロークや一つの編集操作を一単位として履歴化する必要があります。

例えば、ドラッグ中に 100 個のピクセルを変更した場合、pointermove ごとに 100 件の Undo 履歴を作るのではなく、pointerdown から pointerup までを一つの操作として保存する方が自然です。

履歴対象には、次のような操作が含まれます。

- 描画ストローク
- 消去

- 塗りつぶし
- 選択範囲移動
- 貼り付け
- レイヤー追加・削除・並べ替え
- フレーム追加・削除・複製
- パレット変更
- キャンバスサイズ変更

既知の課題として、選択範囲を移動したあと範囲外をクリックして選択解除すると、移動結果が消える場合があります。これは選択中の一時状態と確定済みピクセルデータの境界が曖昧な場合に起こる典型的な問題です。選択移動は、プレビュー表示、移動確定、キャンセルの3状態を明確に分離する必要があります。

9. レイヤーシステム

レイヤーは、一枚の画像を複数の透明な層に分けて管理する仕組みです。背景、キャラクター、装飾、影などを分離して描くことで、他の部分を壊さずに編集できます。

確認できている主な機能は次のとおりです。

- レイヤー追加
- レイヤー削除
- レイヤー順変更
- 表示・非表示
- 不透明度変更
- 合成モード
- フレームごとのレイヤー管理

描画時は、現在選択中のレイヤーだけを編集し、表示時は複数レイヤーを合成します。

記事では、レイヤー複製、ロック、透明部分ロック、フォルダー、クリッピングなど、実装確認できていない高度機能を既存機能として書かないでください。これらを扱う場合は「今後追加できる候補」と表現します。

10. フレームアニメーションシステム

PiXiEEDraw はフレーム単位のアニメーション制作に対応しています。各フレームに画像状態を持ち、それを一定速度で順番に表示することでアニメーションを作ります。

主な機能は次のとおりです。

- フレーム追加

- フレーム削除
- フレーム複製
- フレーム並べ替え
- アニメーション再生・停止
- FPS 設定
- オニオンスキン
- GIF 書き出し

FPS は 1 秒間に表示するフレーム数です。例えば 8FPS なら、8 枚のフレームを 1 秒で表示します。

オニオンスキンとは、現在フレームの前後を薄く重ねて表示する機能です。キャラクターの手足がどの位置からどこへ移動したかを確認しやすくし、歩行、攻撃、点滅などのアニメーション制作を補助します。

記事では、フレームごとの個別表示時間、タグ、ピンポン再生、複数アニメーション範囲などは、実装確認できていないため既存機能として断定しないでください。

11. カラモードとパレット管理

PiXiEEDraw では、インデックスカラーと RGB カラーを扱う設計があります。

インデックスカラーでは、作品内で使用できる色をパレットに限定し、各ピクセルはパレット内の色番号として扱います。限られた色数で統一感を出しやすく、レトロゲーム風の制作に向いています。

RGB カラーでは、パレットに存在しない色も扱いやすく、画像取り込みや自由な色編集に向いています。

確認できているパレット関連機能は次のとおりです。

- プリセットパレット
- カスタムパレット
- パレット色の編集・追加
- PNG・GIF 読み込み時の使用色抽出
- 256 色を超える画像の減色
- インデックスカラーでの近似色変換
- RGB カラーでの不足色追加

PNG または GIF を読み込んだ場合、画像で実際に使われている色を抽出し、カスタムパレットを生成します。色数が 256 色を超える場合は減色して取り込みます。

貼り付け時は、インデックスカラーでは既存パレットの近似色へ合わせ、RGB カラーでは必要な色をパレットへ追加する動作があります。プリセットパレットを編集した場合は、元のプリセットそのものを書き換えるのではなく、カスタムパレットとして扱う設計です。

12. 画像・プロジェクトの読み込み

読み込み対象として確認できているものは次のとおりです。

- PiXiEEDraw 専用プロジェクトファイル (.pixieedraw)
- PNG
- GIF
- クリップボード画像
- PiXiEELENS から渡された画像

専用プロジェクトファイルは、完成画像だけでなく、編集を再開するためのレイヤー、フレーム、パレット、設定などを保存するための形式です。

画像ファイルを読み込む場合は、画像を単純に表示するだけでなく、ピクセルデータへの変換、サイズ確認、色抽出、減色、パレット生成などを行います。

PiXiEELENS から取り込む場合は、現在の作品を直接上書きするのではなく、別プロジェクトタブとして開く設計が確認されています。元の作品を残したまま、カメラ画像のドット化結果を編集できます。

13. 書き出しシステム

確認できている書き出し形式は次のとおりです。

- PNG
- GIF
- JPEG
- SVG
- PiXiEEDraw 専用プロジェクトファイル

PNG は透過を保った静止画やゲーム素材に向いています。GIF はフレームアニメーションの共有に使えます。JPEG は透過が不要な軽量画像向けです。SVG は拡大可能な形式として利用できます。専用プロジェクトファイルは再編集用です。

書き出し画面には、形式選択、出力倍率、原寸の同時出力、画像と一緒にプロジェクトファイルを保存する設定、保存後にコンテスト投稿画面へ移動する設定などがあります。

確認できているオプションには次のものがあります。

- 出力倍率の指定
- 拡大出力時に原寸も追加出力
- 画像・GIF・SVG 出力時に PiXiEED ファイルも同時保存
- 保存完了後にコンテスト投稿画面へ移動
- PNG のグリッド分割設定

グリッド分割は、スプライトシートなどを指定幅・高さで分割出力する用途に関係します。記事では分割順などの詳細を説明する場合、画面表記と実際の出力結果を再確認してください。

14. 端末内自動保存

PiXiEEDraw は、制作中のプロジェクトを端末内へ自動保存し、次回起動時に復元できる仕組みを持ちます。

自動保存は、描画のたびに同期的に大きなデータを書き込むと操作が重くなるため、一定時間の遅延を設けて連続変更をまとめる方式が適しています。ローカル拡張のサンプルでも、変更後に短いタイマーを設定し、連続操作をまとめて保存しています。

本体側でも概念的には次の流れになります。

28. プロジェクトに変更が発生
29. 未保存状態を記録
30. 短い待機時間を設定
31. 追加変更があればタイマーを更新
32. 編集が落ち着いた時点で端末内へ保存
33. 保存成功後に未保存状態を解除

保存対象は完成画像だけではなく、フレーム、レイヤー、パレット、設定、現在位置などを含むプロジェクト状態です。

ブラウザストレージには容量や端末依存性があるため、重要な作品は専用プロジェクトファイルとして書き出す必要があります。端末内自動保存はバックアップではなく、作業継続を支える復元機能として説明するのが適切です。

15. リアルタイム共同編集の概要

PiXiEEDraw の大きな特徴が、複数人で同じプロジェクトを編集するリアルタイム共有モードです。

共有モードでは、主に次の立場があります。

- マスター：共有プロジェクトを開始・管理する側
- 参加者：招待情報を使って共有プロジェクトへ参加する側

基本的な利用の流れは次のとおりです。

34. マスターが共有部屋または共有プロジェクトを作成
35. 招待キーまたは招待 URL を発行
36. 参加者が共有プロジェクトへ接続
37. 現在のプロジェクト状態を取得
38. 以降の変更をリアルタイム購読

39. 各参加者の操作を全員へ反映

共有の難しさは、単に同じ画像を表示することではありません。複数人が同時に操作し、通信が一時的に途切れ、スマートフォンがスリープし、復帰後に未取得の変更が存在する状況でも、データを壊さず追いつく必要があります。

16. Supabase を使った共有基盤

共有機能には Supabase が使用されています。主な役割は次のとおりです。

- ユーザー認証
- 共有プロジェクト情報の保持
- 操作履歴または変更データの保存
- Realtime による変更通知
- 復帰時の不足操作取得

Realtime は、新しい変更が発生したことを参加者へ素早く通知するために使います。ただし、Realtime 通知だけを唯一の正解にすると、切断中の通知を失った場合に同期が崩れます。

そのため、PiXiEEDraw では Revision や操作履歴を使い、「現在どこまで適用済みか」を確認しながら不足分を取得する考え方が重要です。

Realtime は高速な通知経路、データベース上の操作履歴と Revision は復元可能な記録経路として役割を分ける構成が適しています。

17. Revision と操作履歴による同期

共有プロジェクトでは、各変更順に順序を示す Revision を付けます。Revision は「このプロジェクトで何番目の変更か」を表す連番に近い概念です。

参加者は、自分が最後に適用した Revision を保持します。新しい操作が届いた場合、次の Revision であれば適用し、番号が飛んでいる場合は不足分を取得します。

例：

- 現在適用済み：Revision 120
- Realtime で届いた操作：Revision 121
- 連続しているため、そのまま適用

別の例：

- 現在適用済み：Revision 120
- Realtime で届いた操作：Revision 124
- 121～123 を取りこぼしている可能性がある

- サーバーから 121 以降を取得して順番に適用

開発ログ上では、draw-applied、ops-apply、subscribe、watchdog-catchup-stalled、wake-recovery、restore-shared-project などの処理名が使われています。これは、描画反映、操作適用、購読、追いつき監視、復帰処理、起動時復元といった複数の同期経路が存在することを示します。

記事では「Google ドキュメントと完全に同じ方式」など、確認できない比較は避けてください。「操作履歴と Revision を用いて不足分を追跡する設計」と説明するのが安全です。

18. ローカル操作の送信フロー

共有モード中にユーザーが描画した場合、概念的には次の処理を行います。

40. ローカルの対象レイヤーへ描画を適用
41. 操作内容を共有用データへ変換
42. 操作 ID や対象フレーム・レイヤーを付与
43. サーバーへ送信
44. サーバー側で Revision を確定
45. Realtime で参加者へ通知
46. 各参加者が未適用操作として受信
47. Revision 順に適用
48. キャンバスを再描画

自分自身が送信した操作も、サーバーから戻ってくる可能性があります。そのため、操作 ID や操作者 ID を使って二重適用を防ぐ必要があります。

通信が失敗した場合は、未送信操作として再送するか、サーバー側の状態と照合する必要があります。ただし、具体的なオフラインキュー仕様は未確認のため、記事では断定しないでください。

19. 参加者側の操作適用

他の参加者から操作を受け取った場合、対象フレーム、レイヤー、座標、色などを確認し、ローカルのプロジェクト状態へ反映します。

操作適用時には次の点が重要です。

- 同じ操作を二重に適用しない
- Revision 順を守る
- 対象フレームやレイヤーが存在するか確認する
- レイヤー追加・削除など構造変更との順序を守る
- 適用後に必要な領域だけ再描画する
- 自動保存や復元用状態を更新する

大量の操作を一件ずつ画面更新すると重くなるため、複数操作をまとめて適用し、最後に一度再描画する処理が有効です。

ops-apply start/done のようなログは、操作適用処理の開始と完了を追跡し、途中で止まっている場所を特定するために使われます。

20. Watchdog と追いつき理

Realtime 接続が見かけ上つながっていても、新しい操作が届かない、または適用が止まる場合があります。そのため、定期的に同期状態を確認する Watchdog が必要です。

Watchdog は概念的に次を確認します。

- 共有プロジェクトへ参加中か
- 現在の Revision はいくつか
- サーバー側により新しい Revision があるか
- 最後に操作を受信・適用した時刻
- 操作適用処理が長時間止まっていないか
- 再接続または不足操作取得が必要か

watchdog-catchup-stalled は、追いつき処理が進んでいない状態を検出するログ名です。

ただし、起動時復元や Wake Recovery の最中に Watchdog が同時実行されると、同じ操作取得や状態変更が競合する可能性があります。そのため、復元中フラグや共有プロジェクト ID の有無を確認し、必要な Watchdog 処理を抑止する設計が必要です。

21. スリープ・バックグラウンド復帰

スマートフォンや PWA では、アプリをバックグラウンドへ移動したり画面を消したりすると、JavaScript 処理や Realtime 接続が停止・切断されることがあります。

復帰時には、単純に再購読するだけでは不十分です。停止中に他の参加者が操作している可能性があるため、次の処理が必要です。

49. ページが再び表示状態になったことを検知
50. 現在共有プロジェクトへ参加中か確認
51. 起動時復元処理と競合していないか確認
52. Realtime 接続状態を確認
53. 必要に応じて再購読
54. 最後に適用した Revision を確認
55. サーバーから未適用操作を取得
56. Revision 順に適用

57. キャンバスと UI を再描画

58. 編集可能状態へ戻す

この処理が Wake Recovery です。

既知の課題には、PWA 復帰後に描画が即時反映されない、参加者側で同期がずれる、復元処理と Watchdog が競合する、復帰後に描画できなくなる場合がある、といったものがあります。

記事では、共有機能が完全無欠であるとは書かず、「スリープ復帰や再接続も考慮した同期処理を実装・改善している」と表現するのが正確です。

22. 起動時の共有プロジェクト復元

共有プロジェクトを開いた状態でブラウザを閉じた場合、次回起動時に共有状態を復元する処理があります。

起動時復元では次の順序が重要です。

59. ローカルに保存された共有プロジェクト情報を取得

60. 認証状態を確認

61. 共有プロジェクトが現在も有効か確認

62. 初期状態または基準データを取得

63. 保存済み Revision 以降の操作を取得

64. 操作を適用

65. Realtime を購読

66. 復元完了フラグを解除

67. Watchdog を開始

復元中に通常の Wake Recovery や共有更新ループが動くと、二重取得や状態の上書きが起きます。開発時には、起動時復元中であることを示すフラグを導入し、復元中の Watchdog や Wake Recovery を抑止する方針が取られています。

復元処理にはタイムアウトがありますが、大きな共有プロジェクトや通信状況によって時間がかかるため、タイムアウト値の調整も行われています。

23. 競合とデータ整合性

複数人が同じ座標をほぼ同時に編集した場合、どちらを最終状態にするかという競合が発生します。

PiXiEEDraw の具体的な競合解決規則は資料だけでは完全に確定できません。一般的には、サーバー側で確定した Revision 順に操作を適用し、後の操作が最終結果になる方式が考えられます。

ただし、レイヤー削除とそのレイヤーへの描画が同時に発生するような構造的競合は、単純な後勝ちだけでは処理できません。対象の存在確認、無効操作のスキップ、再取得、エラー表示などが必要です。

記事では「競合を完全自動で解決する」と断定せず、「操作順と Revision を基準に整合性を保つ設計」と表現してください。

24. PWA とモバイル対応

PiXiEEDraw には Web App Manifest と Apple Touch Icon が設定され、ホーム画面へ追加して PWA に近い形で利用できます。

PWA 化の利点は次のとおりです。

- ブラウザのタブを探さず起動できる
- アプリに近い全画面表示ができる
- スマートフォンから素早く制作を再開できる

一方で、モバイルブラウザには次の課題があります。

- アドレスバー表示・非表示による画面高の変化
- 画面回転
- セーフエリア
- ソフトキーボード表示
- タッチスクロールと描画操作の競合
- バックグラウンド時の処理停止
- メモリ不足によるページ再読み込み

そのため、viewport-fit、touch-action、Pointer Events、復帰時再レイアウト、Wake Recoveryなどを組み合わせて対応します。

記事では「オフラインですべて使える」とは書かないでください。端末内編集の一部は継続できても、共有編集や認証などは通信を必要とします。

25. ローカル拡張システム

PiXiEEDraw には、JavaScript ファイルを読み込んで補助ツールを追加するローカル拡張機能があります。

拡張は本体ページへ直接コードを注入するのではなく、sandbox iframe 内で動作します。これにより、拡張が本体 DOM や共有データへ自由にアクセスすることを防ぎます。

拡張から利用できる主な API は次のとおりです。

- `api.on(name, handler)`
- `api.toast(message, level)`
- `api.ui.getRoot()`
- `api.ui.clear()`

- `api.ui.mount(node)`
- `api.ui.append(node)`
- `api.ui.setTitle(text)`
- `api.ui.setSubtitle(text)`
- `api.ui.setStatus(text, kind)`
- `api.ui.addStyles(css)`
- `api.ui.el(tag, props, children)`
- `api.getContext()`
- `api.storage.get(key, fallback)`
- `api.storage.set(key, value)`
- `api.storage.remove(key)`

代表的なイベントは `init`、`context`、`interval` です。

ローカル拡張は次の用途に向いています。

- 独自の補助 UI
- 独自 Canvas
- 制作支援計算
- プレビュー
- 端末内設定保存
- スプライト生成補助

一方、現在の安全制約では次の操作はできません。

- 外部通信
- 親ページ DOM の直接編集
- 共有データの直接変更
- 他ユーザー端末への反映
- メインキャンバス上への直接重ね描き
- 本体キャンバス入力の横取り

この制約により、拡張は本体を壊しにくい一方、本体プロジェクトとの深い連携には専用 API の追加が必要です。

26. ローカル拡張の保存と実行

拡張は単一の JavaScript ファイルとして読み込む構成です。外部ライブラリや外部通信に依存せず、拡張パネル内で完結することが基本方針です。

拡張内の状態は `api.storage` を使って端末内へ保存できます。例えば、8 方向スプライト補助拡張では、4 方向のグリッドデータを短い遅延後に保存し、次回読み込み時に復元します。

処理の例は次のとおりです。

68. init イベントを受信
69. api.storage から保存状態を取得
70. 独自 UI と Canvas を構築
71. ユーザー入力で拡張内状態を更新
72. 描画・プレビューを再生成
73. api.storage へ状態を保存
74. 必要に応じて PNG などを書き出す

拡張は共有同期されず、その端末だけで動作します。記事では「拡張を入れると共同編集の全員に機能が追加される」とは書かないでください。

27.8 方向スプライト生成張の例

ローカル拡張の試作として、8方向スプライトを補助するツールがあります。

基本構成は次のとおりです。

- 正面
- 右
- 背面
- 左

の4方向を手描きし、

- 右前
- 右後
- 左後
- 左前

の4方向を自動生成します。

試作版では24×24グリッドを使い、4つの編集 Canvas を同時表示します。右方向を反転して左方向へコピーする機能、サンプル適用、全消去、端末内保存、8方向 PNG またはスプライトマップ出力があります。

斜め方向は、隣接する2方向の形状情報を補間して作ります。例えば右前は正面と右、右後は右と背面を基に生成します。

自動生成結果は完成品を保証するものではありません。4方向の輪郭差が大きい場合は不自然になるため、手直しを前提とした制作補助です。

この拡張は現状、本体のレイヤーやフレームを直接読み取るのではなく、拡張内部の独自 Canvas で動作します。

28. PiXiEELENS との連携

PiXiEELENS は、カメラ画像をドット絵化し、その結果を PiXiEEDraw で編集するための関連ツールです。

利用の流れは次のとおりです。

75. PiXiEELENS でカメラを起動
76. 写真を撮影または画像を取得
77. ドット化处理
78. 結果を PiXiEEDraw へ送る
79. 別プロジェクトとして開く
80. レイヤー、パレット、描画ツールで調整

別プロジェクトとして開くことで、現在編集中の作品を上書きせず、元画像と変換結果を分けて管理できます。

減色方式、ディザリング、背景除去、輪郭強調などの詳細は本資料では確定できないため、記事で既存機能として断定しないでください。

29. 認とアカウント

PiXiEEDraw では Supabase 認証を利用し、Google ログインの導入・利用が検討または実装されています。

アカウントは主にオンライン機能と結びつきます。

未ログインでも使える機能の想定

- 通常の描画
- 端末内保存
- ファイル読み込み・書き出し
- ローカル拡張

ログインが関係する機能の想定

- 共有プロジェクト
- ユーザー識別
- コンテスト投稿
- クラウド関連機能
- 購入・サポート状態

ただし、各機能のログイン必須条件は変更される可能性があります。記事では、実際の公開画面を確認したうえで記述してください。

30. 分析・広告・運営機能

PiXiEEDraw には Google Analytics 系の利用計測と Google AdSense 広告が組み込まれています。

計測では、ページの表示だけでなく、PiXiEEDraw プロジェクトを開いたことを示すイベントが送信される構成です。これにより、訪問数、利用開始、継続率、機能利用状況などの改善判断に使えます。

書き出し画面には広告枠が存在します。無料提供を維持するための収益源ですが、広告だけで開発・サーバー費用を賄うのは難しく、広告非表示、支援プラン、有料拡張、素材販売なども検討されています。

記事で料金や有料プランを扱う場合は、最新の公開状態を確認してください。過去に検討した価格案を現在の正式価格として記載してはいけません。

31. 知の課題

開発上、特に重要な課題は次のとおりです。

共有同期

- 参加者の描画が即時反映されない場合がある
- PWA 復帰後に Revision 差が生じる場合がある
- Realtime 再接続後に描画できなくなる場合がある
- 起動時復元、Wake Recovery、Watchdog が競合する可能性がある
- 大きな共有プロジェクトの復元がタイムアウトする可能性がある

編集

- 選択範囲移動後、選択解除時に移動内容が消える場合がある
- 一時選択状態と確定状態の分離が必要

モバイル

- スリープ復帰後の接続・UI 復元
- URL バーや画面高変化によるレイアウトずれ
- タッチ操作とスクロールの競合

操作性

- ブラシサイズを制作中にすぐ変更できる常設 UI
- 書き出し時のファイル名指定
- 初心者向け導線とヘルプ

拡張

- 本体キャンバスやレイヤーへ安全にアクセスする API が不足
- 拡張の状態は共有されない

これらは、製品の欠点を並べるためではなく、システム設計を理解し、今後の改善点を説明するために記載しています。

32. システム上の強み

PiXiEEDraw のシステム上の強みは、単独の機能ではなく、複数の仕組みを一つの制作体験にまとめている点です。

- インストール不要の Web アプリ
- PC・タブレット・スマートフォン対応
- 端末内自動保存と起動時復元
- レイヤーとフレームアニメーション
- PNG、GIF、JPEG、SVG、専用形式の出力
- パレット抽出と減色
- リアルタイム共同編集
- Revision と操作履歴による再同期
- PWA のスリープ復帰対策
- sandbox 型ローカル拡張
- PiXiEELENS など関連ツールとの連携

特に、共同編集とローカル拡張は、一般的な軽量ブラウザドット絵ツールとの差別化につながります。

33. システム上の制約

Web ブラウザ上で動作するため、デスクトップ専用アプリと比べて制約もあります。

- ブラウザストレージ容量に制限がある
- 端末やブラウザによって動作差がある
- バックグラウンドで接続が止まることもある
- 大規模キャンバスや大量フレームではメモリ負荷が高くなる
- ファイルシステムへ自由にアクセスできない
- ローカル拡張は安全のため本体へ直接アクセスできない
- ネットワーク状況が共有編集へ影響する

PiXiEEDraw は、これらを端末内保存、専用ファイル、復帰処理、Revision 同期、sandbox 拡張などで補っています。

記事では、デスクトップアプリよりすべての面で優れていると主張するのではなく、「ブラウザであることの利便性と制約を理解した設計」と説明する方が信頼性があります。

34. 今後追加が考えられる機能

以下は将来構想または推奨案であり、現在の実装済み機能として扱ってはいけません。

- クラウドプロジェクト保存
- レイヤーフォルダー
- レイヤーロック・透明部分ロック
- 高度な選択変形
- アニメーションタグ
- フレームごとの個別時間
- ピンポン再生
- 高度なスプライトシート設定
- 本体プロジェクトへアクセスできる拡張 API
- 拡張マーケット
- 共同編集用コメント・カーソル表示
- AI による補間・配色・アニメーション補助
- 8 方向スプライトの本体統合
- 3D 素体やポーズ参考との連携
- 素材販売・ライセンス管理

これらを記事で扱う場合は、「目指している」「検討している」「将来的に追加可能」と書いてください。

35. GPT に記事を書かせる際の事実ルール

GPT へ記事作成を依頼する際は、次のルールを与えると誤情報を減らせます。

- 本資料に「実装済み」と明記されていない機能を、現在利用できる機能として書かない
- 内部変数名やデータベース項目名を推測して断定しない
- 共有同期を「絶対にずれない」と表現しない
- PWA を「完全オフライン対応」と表現しない
- 8 方向生成を「AI が完成品を自動作成」と表現しない
- ローカル拡張が本体や他人の端末を操作できると書かない
- Aseprite の完全な代替と断定しない
- 料金や有料プランは最新情報を確認する
- 競合解決方式の詳細は、確認できない場合「Revision 順に整合性を保つ設計」と表現する
- 不明点は創作せず「開発中」「未確認」とする

記事の価値は、機能数を誇張することではなく、ブラウザ上で共同制作まで行える設計思想と、そこに必要な復元・同期処理を分かりやすく伝えることにあります。

36. GPT 記事作成用の推プロンプト

以下の文面を、本資料と一緒に GPT へ渡してください。

あなたは、ソフトウェアと Web アプリの仕組みを初心者にも分かりやすく説明できる技術ライターです。

添付した「PiXiEEDraw システム詳細資料」を唯一の基礎資料として、PiXiEEDraw を紹介する記事を書いてください。

条件：

- 資料にない機能を創作しない
- 実装済み、開発中、将来構想を明確に分ける
- 専門用語は最初に簡単に説明する
- 宣伝だけでなく、なぜその仕組みが必要かを説明する
- リアルタイム共同編集では、Realtime、操作履歴、Revision、Wake Recovery の役割を分けて説明する
- ローカル拡張は sandbox iframe 内で動き、本体 DOM や共有データへ直接アクセスできないことを書く
- 読者はドット絵制作者、個人ゲーム開発者、Web アプリ開発に興味がある人を想定する
- 誇張表現を避ける
- 事実不明な箇所は断定しない

記事構成：

1. PiXiEEDraw とは何か
2. ブラウザでドット絵を描く仕組み
3. レイヤーとフレームアニメーション
4. 自動保存と復元
5. リアルタイム共同編集の仕組み
6. スリープ復帰と同期ずれへの対策
7. ローカル拡張の安全設計
8. 現在の課題と今後の展望
9. まとめ

文体：

- 日本語
- 丁寧だが硬すぎない
- 見出しを付ける
- 4,000～6,000 文字
- 技術者以外にも読める内容
- 最後に PiXiEEDraw を試す自然な導線を入れる

37. 記事用の短い要約

PiXiEEDraw は、ブラウザ上でドット絵の制作、レイヤー編集、フレームアニメーション、複数形式の書き出し、リアルタイム共同編集を行える Web アプリです。

制作データは端末内へ自動保存され、起動時に復元できます。共有モードでは Supabase Realtime を使って変更を通知しながら、操作履歴と Revision によって取りこぼした変更を追跡します。スマートフォンのスリープや PWA 復帰後には Wake Recovery を行い、不足している操作を再取得して同期状態へ戻します。

また、JavaScript ファイルを読み込むローカル拡張機能を持ちます。拡張は sandbox iframe 内で動作し、本体 DOM、共有データ、他ユーザーの端末へ直接アクセスできない安全設計です。8 方向スプライト生成など、独立した制作補助ツールを追加できます。

PiXiEEDraw は、ブラウザの手軽さだけでなく、共同制作、復元性、拡張性を組み合わせたドット絵制作環境を目指しています。

38. 情報の基準日と注意

本資料の基準日は 2026 年 6 月です。

資料は、2026 年 3 月時点の PiXiEEDraw 公開 HTML、保存されているローカル拡張コード、これまでの開発相談、共有同期ログ、機能改善方針を基に整理しています。

PiXiEEDraw は継続開発中のため、画面、機能、料金、保存形式、共有仕様は今後変更される可能性があります。公開記事を作成する直前に、実際の PiXiEEDraw 画面と最新の公式説明を確認してください。